Práctica 1: Introducción a R y RStudio

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es introducir R y el entorno de desarrollo RStudio. Al finalizarla, el estudiante será capaz de ejecutar instrucciones en la consola, realizar cálculos elementales, utilizar variables y organizar sus primeros programas en ficheros `.R`.

2. Primeros pasos con R

2.1. Valores numéricos

Entre otras cosas, utilizaremos R para realizar cálculos numéricos. Tenemos que entender que los ordenadores tienen una capacidad de representación de números que está limitada al número de bits que emplean para representar estos números. Los ordenadores representan los números con precisión finita. En particular, los enteros pueden representarse de forma exacta dentro del rango permitido por el tipo de datos. Declaramos los números enteros en R con la sintaxis `nL`, siendo `n` el número que queremos representar y `L` el sufijo que indica que se trata explícitamente de un entero. Así pues, los números enteros 2, 8, -5, 15294 se escriben en el lenguaje R como:

```
2L          8L          -5L          15294L
```

Los números con parte decimal se representan mediante números de punto flotante. Su representación tiene precisión finita, por lo que no todos los números reales pueden representarse exactamente; por ejemplo, π solo puede almacenarse mediante una aproximación. Los números 2.0, 8.0, -5.4, 15294.123 se escriben en el lenguaje R como:

```
2.0         8           -5.4         15294.123
```

Observa que `8` y `8L` representan el mismo valor numérico, pero pertenecen a clases diferentes: `8` es `numeric` y `8L` es `integer`.

En R los enteros se conocen como `integer`, mientras que los números con decimales se llaman `numeric`. Podemos consultar la clase de un objeto `x` mediante la función `class(x)`. Por ejemplo:

```
> class(5)
```

```
[1] "numeric"
```

```
> class(3.5)
```

```
[1] "numeric"
```

```
> class(3L)
```

```
[1] "integer"
```

La constante π viene implementada en R. La forma de escribir en R dicha constante es `pi`:

```
> pi
```

```
[1] 3.141593
```

Idea clave. En esta práctica no necesitamos memorizar los detalles internos de la representación numérica. Lo importante por ahora es distinguir entre valores `numeric` e `integer` y saber consultar su clase con `class()`.

2.2. Operaciones aritméticas básicas

El siguiente paso es el de realizar operaciones aritméticas. Las operaciones aritméticas básicas de R son las siguientes:

Operación	**Símbolo en R**
Suma	`+`
Resta	`-`
Multiplicación	`*`
División	`/`
Potencia	`^`
Módulo (resto de la división entera)	`%%`
División entera	`%/%`

Además de estos operadores, podemos usar los paréntesis `(` y `)` para alterar el orden de resolución de las operaciones matemáticas. R sigue la jerarquía estándar de operaciones, que ordenadas de mayor prioridad a menor prioridad, es la siguiente:

1. Paréntesis: `( )`
2. Potencia: `^` o `**`
3. Multiplicación y división: `*` y `/`
4. Suma y resta: `+` y `-`

Podemos hacer los cálculos directamente en la consola de R. Por ejemplo, si queremos hacer el cálculo $1 + 1$, es suficiente con escribir en la consola lo siguiente:

```
> 1+1
```

Obtendremos la salida siguiente, que nos informa de que el resultado obtenido es 2.

```
[1] 2
```

Podemos hacer otras operaciones:

```
> 2*(3+1)
```

```
[1] 8
```

```
> 7^5
```

```
[1] 16807
```

Ejercicio 1. Realiza los siguiente cálculos en la consola de R:

a) $17 + 8$, $25 - 37$ y $6 \cdot 9$.

b) $\frac{45}{8}$, 2^8 y $7^3 - 4^2$.

c) Calcula `37%%5` y `37%/% 5`. ¿Qué diferencia observas?

d) $2(3+5)^2$ y $2 \cdot 3 + 5^2$. ¿Por qué los resultados son diferentes?

2.3. Valores booleanos

En lógica proposicional, las proposiciones pueden tomar uno de dos valores: verdadero o falso. Estos valores se conocen como *valores booleanos*. En R estos valores se representan como `TRUE` y `FALSE`.

2.4. Operaciones relacionales

Es habitual que necesitemos comparar valores para determinar si son iguales, distintos, o si uno es mayor o menor que otro. Estos operadores, llamados *operadores relacionales*, son esenciales a la hora de escribir nuestro código. En R tenemos los siguientes operadores relacionales:

Operador	Descripción	Ejemplo	Resultado
`==`	Igual a ($=$)	`5 == 3`	`FALSE`
`!=`	Diferente de ($\neq$)	`5 != 3`	`TRUE`
`<`	Menor que ($<$)	`3 < 5`	`TRUE`
`<=`	Menor o igual que ($\leq$)	`5 <= 3`	`FALSE`
`>`	Mayor que ($>$)	`5 > 3`	`TRUE`
`>=`	Mayor o igual que ($\geq$)	`5 >= 3`	`TRUE`

Ejercicio 2. Realiza los siguiente cálculos en la consola de R:

a) Comprueba si $7 = 7$, si $7 = 8$ y si $7 \neq 8$.

b) Determina si $3 < 5$, $10 \geq 10$ y $4 > 9$.

c) Compara las expresiones $2 + 3$ y 5, y también 2^3 y 3^2.

d) Prueba `5 == 5L` y `5 != 5L`. ¿Qué observas?

2.5. Operaciones lógicas

En numerosas ocasiones necesitaremos combinar valores booleanos mediante operaciones lógicas. En R tenemos las siguientes operaciones lógicas:

Operador de R	Descripción	Ejemplo	Resultado
`&&`	AND lógico ($\wedge$)	`TRUE && FALSE`	`FALSE`
`||`	OR lógico ($\vee$)	`TRUE || FALSE`	`TRUE`
`!`	NOT lógico ($\neg$)	`!TRUE`	`FALSE`
`xor()`	OR exclusivo (XOR)	`xor(TRUE, TRUE)`	`FALSE`

Ejercicio 3. Realiza los siguiente cálculos en la consola de R:

a) Evalúa `TRUE && TRUE`, `TRUE && FALSE`, `TRUE || FALSE` y `FALSE || FALSE`.

b) Evalúa `!TRUE` y `!FALSE`.

c) Calcula `xor(TRUE, FALSE)` y `xor(TRUE, TRUE)`.

d) Escribe una expresión que compruebe si 12 está entre 5 y 20, incluyendo ambos extremos. Puedes combinar varios operadores lógicos.

e) Escribe una expresión que compruebe si 5 es o bien menor que 3 o bien menor que 7.

2.6. Funciones básicas

Una de las ventajas de R para realizar cálculos matemáticos es que incorpora numerosas funciones matemáticas de uso habitual.

Función de R	Ejemplo	Resultado
`sqrt(x)`	`sqrt(16)`	`4`
`log(x)`	`log(10)`	`2.302585`
`log10(x)`	`log10(100)`	`2`
`exp(x)`	`exp(1)`	`2.718282`
`sin(x)`	`sin(pi/2)`	`1`
`cos(x)`	`cos(pi)`	`-1`
`abs(x)`	`abs(-5)`	`5`
`round(x, n)`	`round(3.14159, 2)`	`3.14`

Por ejemplo, podemos hacer otras operaciones:

```
> sqrt(2)
```

```
[1] 1.414214
```

```
> abs(-3.5)
```

```
[1] 3.5
```

Ejercicio 4. Realiza los siguientes cálculos en la consola de R y anota el resultado obtenido:

1. Calcula $\sqrt{3}$, $\sqrt{144}$ y $\sqrt{3{,}6}$. ¿Qué pasa si intentas calcular $\sqrt{-1}$? Investiga por tu cuenta el significado de `NaN`.

2. Calcula $\log(1)$, $\log(0{,}001)$ y $\log(47)$. ¿Qué pasa si intentas calcular $\log(-14{,}3)$? ¿Y $\log(0)$? Investiga por tu cuenta el significado de `Inf`.

3. Calcula $\log_{10}(1)$, $\log_{10}(0{,}001)$ y $\log_{10}(10000)$. ¿Qué pasa si intentas calcular $\log_{10}(0)$? ¿Y $\log_{10}(-10)$?

4. Calcula e^0, e^1, $e^{10{,}5}$ y e^{-2}. ¿Qué pasa si intentas calcular $\log(e^5)$? ¿Y $e^{\log(3)}$?

En otros lenguajes es habitual que muchas de estas funciones estén implementadas en paquetes externos. Por ejemplo, en `C` la mayoría de estas funciones se encuentran en la librería `math.h`, en `C++` se encuentran en `cmath` o en el espacio de nombres `std`, en `Java` se encuentran en la clase `java.lang.Math`, y en `Python` se encuentran en el módulo `math`. No es necesario que controles (por el momento) todos los términos que hemos introducido en este párrafo.

3. Uso de variables

Una variable es un nombre asociado a un valor que podemos reutilizar en otros cálculos y, si es necesario, actualizar mediante una nueva asignación. Para crear una variable, escogemos un nombre descriptivo y le asignamos un valor. Para asignarle un valor, utilizaremos la sintaxis

```
mi_variable <- 4096L
```

`<-` indica que estamos realizando una asignación. Por ejemplo:

```
> base <- 4
> altura <- 3
> base*altura/2
```

```
[1] 6
```

Resulta evidente que podemos asignar un nuevo valor a una variable que ya tenía un valor:

```
> base <- 6
> base*altura/2
```

```
[1] 9
```

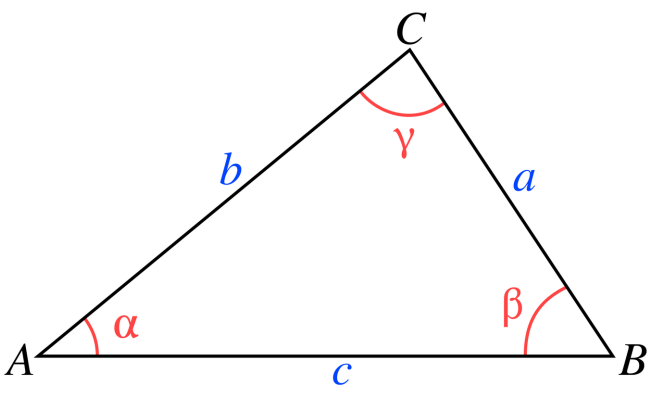
Ejercicio 5. Realiza los siguientes cálculos declarando las variables necesarias:

a) Calcula el área de un círculo de radio $r = 2$.

b) Calcula el volumen de un cilindro de altura $h = 8$ y radio de la base $r = 3$.

c) Calcula la longitud de la hipotenusa de un triángulo rectángulo de catetos $b = 4$ y $c = 3$.

d) Un triangulo como el que se muestra en la imagen tiene lados $a = 5$, $b = 6$ y un ángulo $\gamma = \frac{\pi}{3}$. Cálcula el lado c y los ángulos α y β. Para ello, recuerda los Teoremas del Seno y del Coseno.



e) Define `a <- 7` y `b <- 3`. Calcula y almacena en una variable `c` el valor de $a^2 + b^2$. Comprueba después si `c` es igual a 58.

f) Un producto tiene un precio de 24,50 €. Por la compra de 3 unidades se nos realiza un descuento del 10 %. Calcula el importe total de aprovechar esta oferta.

4. Primeros pasos con RStudio

Una vez familiarizados con R, podemos trabajar con RStudio, un entorno de desarrollo que facilita la escritura, ejecución y organización de código en R.

4.1. Consola, editor y entorno de trabajo

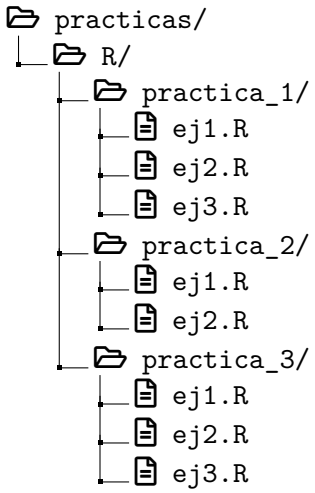
RStudio separa varias tareas que al principio conviene distinguir:

- **Consola:** permite ejecutar instrucciones de R inmediatamente.
- **Editor de scripts:** permite escribir, guardar y modificar programas completos en ficheros `.R`.
- **Entorno de trabajo:** muestra, entre otras cosas, los objetos que hemos creado durante una sesión.

Una forma recomendable de trabajar es **probar una instrucción en la consola, pasarla al script cuando funcione y guardar el script**. Así, el trabajo queda registrado y puede repetirse posteriormente.

4.2. Estructura de directorios

Es recomendable mantener desde el principio una estructura de carpetas sencilla y ordenada. Esto facilitará la localización de los ejercicios y evitará mezclar ficheros de prácticas diferentes. Un ejemplo de cómo podemos estructurar nuestro directorio es el siguiente:



Ejercicio 6. Crea esta estructura de directorios, sin los ficheros `.R`, que los iremos añadiendo sobre la marcha (o una estructura similar que te ayude a organizarte). Observa que este ejercicio se alargará a lo largo de todo el curso.

Iremos ampliando esta estructura conforme vayan pasando las prácticas.

5. Comentarios

Comentar el código es una buena práctica de programación: los comentarios permiten explicar qué hace una parte del programa y señalar aspectos que puedan resultar relevantes. Los comentarios en R vienen precedidos por el carácter `#`.

```
# Un codigo de R con una linea al comienzo
1+1
sqrt(3.6)
# Las lineas de arriba son instrucciones de R
3L + 2L^5L # Tambien podemos poner los comentarios a continuacion de la
    instruccion
```

6. Ejercicios de RStudio

Para los ejercicios realizados en la sección 2, crea uno o varios ficheros `.R` desde RStudio y guárdalos en el directorio correspondiente. Reescribe en el editor el código que has utilizado y ejecútalo desde RStudio. Añade comentarios con las salidas esperadas y una cabecera breve con la información de la práctica. Puedes usar el siguiente ejemplo como inspiración, o buscar un estilo propio.

```
#################################################################
# Autor: Miguel Beltra Vidal                                    #
# Laboratorio de Matematicas - Grado en Ingenieria Matematica   #
# Curso 2026/27                                                 #
#                                                               #
# Practica 1 - Ejercicio 1                                      #
#                                                               #
#################################################################

2^5             #El resultado es 32
(1+1)^(3-1)*6   #El resultado es 24
```

Prueba a ejecutarlo y comprueba que los resultados coinciden con aquellos obtenidos cuando los escribías directamente en la terminal.

7. Entrega y consejos

En la próxima práctica explicaremos cómo entregar los ejercicios resueltos. Por el momento:

1. guarda tus scripts en la carpeta de la práctica correspondiente;
2. utiliza nombres de fichero claros y consistentes;
3. comenta únicamente aquello que ayude a entender el código;
4. comprueba que puedes ejecutar de nuevo el script desde el principio sin depender de instrucciones que hayan quedado solamente en la consola.

Idea clave: un buen script debe ser reproducible, legible y ordenado.